



COMP2017 9017

Assignment 1

Due: 23:59 1 April 2026

This assignment is worth 12.5% of your final assessment

This assessment is CONFIDENTIAL. © University of Sydney

Contents

1	Assignment 1- [PointerPro Animate] - 12.5%	3
2	Introduction	4
3	Background	4
4	Task	4
5	API	5
5.1	[COMP9017 only]	5
5.2	Performance requirements	5
6	Programming guide	6
6.1	Image and movie file conversions	6
6.2	Testing guide	6
7	Short answer questions	7
8	Marking	8
8.1	Compilation requirements	8
8.2	Test structure	8
8.3	Automated submission testing	8
8.4	Marking criteria	9
8.5	Restrictions	9

9	Submission Checklist	10
A	Version history	11
B	API documentation	11

List of Acronyms

API Application Programming Interface. [5](#), [6](#), [8](#)

ASAN Address Sanitiser. [8](#), [9](#)

PIC Position Independent Code. [8](#)

UI User Interface. [4](#), [5](#)

UX User eXperience. [4](#), [5](#)

VLA Variable Length Array. [9](#)

1 Assignment 1- [PointerPro Animate] - 12.5%

We strongly recommend reading this entire document at least twice. You are encouraged to ask questions on Ed after you have first **searched**, and checked for updates of this document. If the question has not been asked before, make sure your question post is of type "**Question**" and is under "**Assignment**" category → "**P1**". Please follow the staff directions for using the question template.

It is important that you continually back up your assignment files onto your own machine, flash drives, external hard drives and cloud storage providers (as private). You are encouraged to submit your assignment regularly while you are in the process of completing it.

Full reproduction steps (seed, description of what you tried) **MUST** be given if you are enquiring about a test failure or if you believe there is a bug in the marking script.

Academic Declaration

By submitting this assignment you declare the following: I declare that I have read and understood the University of Sydney Student Plagiarism: Academic Integrity Policy, Coursework Policy and Procedures, and except where specifically acknowledged, the work contained in this assignment/project is my own work, and has not been copied from other sources or been previously submitted for award or assessment.

I understand that failure to comply with the Student Plagiarism: Academic Integrity Policy, Coursework Policy and Procedures can lead to severe penalties as outlined under Chapter 8 of the University of Sydney By-Law 1999 (as amended). These penalties may be imposed in cases where any significant portion of my submitted work has been copied without proper acknowledgement from other sources, including published works, the Internet, Generative AI where approved, existing programs, the work of other students, or work previously submitted for other awards or assessments.

I acknowledge that I have reviewed and understood the University of Sydney's guidelines on the responsible use of Generative AI ¹ and will adhere to them in accordance with academic integrity policies.

I realise that I may be asked to identify those portions of the work contributed by me and required to demonstrate my knowledge of the relevant material by answering oral questions or by undertaking supplementary work, either written or in the tutorial, in order to arrive at the final assessment mark.

I acknowledge that the School of Computer Science, in assessing this assignment, may reproduce it entirely, may provide a copy to another member of faculty, and/or communicate a copy of this assignment to a plagiarism checking service or in-house computer program, and that a copy of the assignment may be maintained by the service or the School of Computer Science for the purpose of future plagiarism checking.

¹<https://www.sydney.edu.au/students/academic-integrity/artificial-intelligence.html>

2 Introduction

Memory Leak Studios is developing a web-based animation service: *PointerPro Animate*. Their [User Interface \(UI\)/User eXperience \(UX\)](#) team has been working hard on the web interface but they still require performant and reliable animation software. As such, they have contracted you, a highly skilled systems programmer, to develop the backend software for their service.

3 Background

An animation is a sequence of images, known as frames, that are displayed at fixed time intervals. Each frame is a 2D matrix of colour values, with each cell corresponding to a pixel position within the frame. This representation is often referred to as a [bitmap](#)².

Colour values can be expressed in many ways. In an effort to bring the product to market as soon as possible, you have been allowed to limit support to ARGB32³ format input and output. This format comprises three 8b values for individual red, green and blue colour components as well as a fourth 8b value for alpha (often referred to as transparency). Note that these four values fit conveniently into one `uint32_t` value.

Animation frames are constructed by combining a collection of graphical objects known as *sprites*⁴. Sprites may be repositioned in each frame to give the illusion of motion of the sprite throughout the animation. While sprites are generally represented as a small bitmap, they may be constructed programmatically.

Sprites are combined by copying their bitmap representation to the frame at the desired position. Since sprites may overlap, the alpha channel must be considered during the copying process and the order in which sprites are combined will impact the outcome of animation.

4 Task

You will develop an animation generator backend that specialises in efficiently managing sprites, their motion in the frame, and the order in which they are combined.

Sprites will be constructed from either a user-provided bitmap image file in ARGB32 format or a small variety of simple shapes. In the initial release, these shapes will be rectangles (filled or unfilled) and circles (filled only). However, it is expected that the variety of shapes will grow – your implementation should be easily extendable to consider this.

You are required to implement a memory efficient solution. A significant amount of memory may be required to store sprite data, particularly if that sprite is replicated in one animation or used in multiple animations. Sprite data is to be stored only once regardless of the number of times the sprite is placed in the animation(s).

Motion will be configured in the style of a physics engine. Given the initial position p_0 the position

²[Wikipedia: Bitmap](#)

³[Wikipedia: RGBA color model](#)

⁴[Wikipedia: Sprites](#)

in the frame at some time t is calculated as follows:

$$p = p_0 + vt + \frac{at^2}{2} \quad (1)$$

The ability to configure a custom animation engine has been considered but is not required for this release.

The [UI/UX](#) team has provided an [Application Programming Interface \(API\)](#) for seamless integration with their work. You are required to follow this [API](#) in your design. Although best efforts have been made to ensure this [API](#) is complete and correct, you may be required to adjust your implementation based on [API](#) changes communicated by the [UI/UX](#) team.

Your backend service is required to generate animation frames from the provided canvas size, sprites and animation parameters. When generating a frame, the alpha component of bitmap sprites must be considered. You are not required to implement alpha blending; you should consider a *zero* alpha value as 100% transparent and *non-zero* as 100% opaque. That is, a sprite pixel should only be written to the frame if the alpha value is *non-zero*. The colour values in the final frames should have alpha set to its maximum value (0xFF).

5 API

The API documentation is provided with the source in a form that is compatible with Doxygen document generator. A copy can be found in [Appendix B](#).

5.1 [COMP9017 only]

The following functions are only required to be implemented by COMP9017 students:

```
void animate_placement_up(struct sprite_placement*);  
void animate_placement_down(struct sprite_placement*);
```

It is recommended that COMP2017 students also implement these functions, however no marks will be awarded to COMP2017 students for doing so.

5.2 Performance requirements

Memory usage and leaks in your program will be tracked during marking by dynamically replacing symbols `malloc`, `calloc`, `realloc` and `free`.⁵ You should only use the above standard dynamic memory allocation functions.

The following functions, where implementation is required, are expected to have execution time complexity $O(1)$:

```
struct sprite_placement* animate_place_sprite(struct canvas* canvas,  
...);  
void animate_placement_top(struct sprite_placement*);  
void animate_placement_bottom(struct sprite_placement*);
```

⁵Note that some functions, like `printf`, also use dynamic memory. Do not call them in your submission.

```
void animate_placement_up(struct sprite_placement*);  
void animate_placement_down(struct sprite_placement*);
```

For those interested, the [API](#) contains an optional extension if you wish to continue working on this project after final submission:

```
void animate_set_animation_funtion(struct sprite_placement*, ...);
```

This function accepts a pointer to a user-defined function that should be used for calculating the appropriate position of a sprite at any time during the animation.

6 Programming guide

6.1 Image and movie file conversions

This assignment will require you to work with files of various formats. This section provides a guide on how to convert images to the appropriate format and how to convert frames generated by your code to image or video files.

Note: $\{VAR\}$ in this guide implies an environment variable of name “VAR” exists. You must either set an environment variable with the listed name to the appropriate value or replace that token with your desired value directly.

imagemagick can be used to convert images to the format expected by AnimatePro using the following command

```
mogrify -format bmp -depth 8 -alpha on ${INPUT_FILE}
```

imagemagick can be used to convert raw pixel data into an image file using the following command:

```
convert -size ${HEIGHT}x${WIDTH} -depth 8 \  
-color-matrix '0 0 1 0 0 1 0 0 1 0 0 0 0 0 0 1' \  
RGBA:${INPUT_FILE} ${OUTPUT_FILE}
```

ffmpeg can be used to generate the final animation from the raw frame data provided by your software using the following command:

```
ffmpeg -f rawvideo -pix_fmt bgra -s ${HEIGHT}x${WIDTH} -r 60 \  
-i ${INPUT_FILE} -c:v libx264 -y ${OUTPUT_FILE}
```

6.2 Testing guide

It is highly recommended that you develop automated tests for your submission. For COMP2017 students, your tests will not be directly marked, but may be chosen as a short-answer question (Section 7). For COMP9017 students, you will be required to answer that question.

A testing framework has not been prescribed. You are welcome to use any method that you are familiar with. What ever you choose, your test framework should be quick to run to verify that

existing functionality remains correct and issues with the functionality under development are easy to detect.

Tests should aim to test only one feature at a time to narrow the search space for bugs. However, you will find that some features depend on others. In this case, you may wish to skip such tests if the tests for functionality on which they depend are failing.

A simple framework might involve creating independent *main.c* files for each test, or a single *main.c* that accepts one (or many) arguments to select the test and its parameters. A *bash* script could then be used to orchestrate the test, compare results and, most importantly, clearly indicate which of your tests have passed and which have failed.

Visually inspecting video files or image files is time consuming, error prone and cannot be automated. It is best to compare individual byte or pixel values of the generated frames. Inspecting bytes is easy for a computer program, but not for mere humans. You may wish to use a combination of *xxd* and *diff*.

xxd reads binary data and prints that data in hexadecimal form. Since pixel values occupy 4 consecutive bytes, inspection will be easier if we tell *xxd* to group our data into bytes of 4. *xxd* will also take the endianness of the machine into consideration – we can expect the values reported by *xxd* to match values printed in our code.

```
xxd -g4 ${INPUT_FILENAME} | sed 's/  .*/g' > ${OUTPUT_FILENAME}
```

diff reports on difference between two ASCII files. Binary data can still be processed if we first convert that binary data to ASCII using a tool such as *xxd*. When comparing two files (large files in particular) suppressing the printing of common lines allows us to focus only on the differences.

```
diff -y --suppress-common-lines \  
  ${OUTPUT_FILENAME} ${EXPECTED_FILENAME}
```

Tip: keep your frame size small during initial testing. Identifying the pixel position of data reported can be challenging for very large frames.

7 Short answer questions

As part of the in-tutorial code review in week 8, you are required to analyse your code and prepare for **two** of the below questions. You must supplement your answer with references to your code. The examiner will also ask follow-up questions based on your response. COMP9017 students must answer Q4.

Q1: How did you ensure that a sprite could not be freed until all placements for that sprite were destroyed?

Q2: How would you extend your implementation to consider a wider variety of shapes?

Q3: How would you extend your implementation to handle sprite grouping (where a group of sprites could be given the same motion in the animation)?

Q4: Demonstrate how you constructed test cases and the testing methods used to confirm your program functions correctly. If you answer this question, the testcases must be in your final submission in a folder named `tests`, and all tests should be run by the file `tests/run_all_tests.sh`.

8 Marking

8.1 Compilation requirements

Your submission must produce an object file named `animate.o` using the command `make animate.o`. The marking script will compile this into a *shared object*. Thus, the flag `-fPIC` must be added.

You are free to (and encouraged to) add extra build rules and functions for your local testing, such as a `main` function or debug flags. [Address Sanitiser \(ASAN\)](#) is encouraged during local testing, **and will be automatically added to your final submission⁶**.

When marking, your code will be compiled and run entirely on the Ed workspace. The aforementioned `make` commands will be run to compile your program and run the executable. If it does not compile in the Ed environment, then your code will receive **no marks for your attempt**. When submitting your work ensure that any binary files generated are not pushed onto the repository.

8.2 Test structure

After your object file is compiled into a shared object, python scripts using `ctypes` will interact with the functions described in spec. In most cases, the script is responsible for:

- Creating temporary data,
- Orchestrating calling of functions,
- Comparing returned data with expected values.

Thus, you can think of the test inputs and outputs as not given from a separate program, but rather driven in the same program. In this way, your program outputs are validated before your program ends.

8.3 Automated submission testing

After submission, a subset of the tests that will be used for marking will be run on your submission. This is to provide some feedback to you on your progress and on your understanding of the problem specification. The sequence of [API](#) calls will be reported to you such that you can reproduce a similar test in your own framework (and extend that test to improve coverage).

These tests should be used as a guide only. A hidden set of tests will be run against your submission for final marking after the submission deadline has passed.

⁶The marking script will attempt to **add ASAN** and [Position Independent Code \(PIC\)](#) during compilation by appending the flags `-fno-sanitize=all -fPIC -Wvla -Werror -fsanitize=address -g`. If this is not successful, marking will silently fail.

8.4 Marking criteria

The assignment is worth 12.5% of your final grade. This is marked out of 25, and breaks down as follows. For marks awarded per testcase, please refer to Edstem.

Marks	Item	Notes
3/25	Code Style	Manual marking
2/25	Week 8 in-tutorial review*	Manual marking
5/25	Correctly generates frames with 1 sprite	Automatic tests
5/25	Layers sprites correctly on placement	Automatic tests
5/25	Rearranges layers correctly	Automatic tests
2/25	Generates animations correctly	Automatic tests
3/25	O(1) sprite placement/layer operations	Automatic tests

* Later weeks if you have DAP or SCON, we will reach out to arrange a alternative interview time.

For style, refer to the [style guide](#). You will also be marked based on the modularity and organisation of your code. For full marks, code should be organised in multiple source files, and use modular, task-specific functions. Organised data structures are essential here. Style marking is only applied for reasonable attempts at correctness.

8.5 Restrictions

To successfully complete this assignment:

- The code must entirely be written in the C programming language.
- Free all dynamic memory that is used.
- *NOT* declare any global variable.
- *NOT* use any external libraries other than those in `libc`.
- *NOT* use [Variable Length Arrays \(VLAs\)](#).
- Have a clean repository. This means no object, executable, or temporary files **for any commit** in the repository, just your final submission.
- Only include header files that contain declarations of functions and extern variables. Do not define functions within header files.
- Must use meaningful commits and meaningful comments on commits.
- Other restricted functions may come at a later date.
- Any and all comments must be written only in the English language.
- *NOT* manually use return code 42, reserved by [ASAN](#).

Submissions breaking these restrictions will receive a deduction of up to 6 marks *per breach*.

The red flag items below will result in an immediate zero. Negative marks can be assigned if you do not follow the spec or if your code is unnecessarily or deliberately obfuscated:

- Any attempts to deceive or disrupt the marking system.
- Use any of the below functions. You shouldn't need to use these functions at all in your program, and you are doing something terribly wrong if you are.

- `_init`, `atexit(2)`, `_exit(2)`, `_Exit(3)`
- `dlopen(3)`, `dlsym(3)`, `dlclose(3)`
- `fork(2)`, `vfork(2)`, `execve(2)`, `exec*(3)`, `clone(2)`
- `kill(2)`, `tkill(2)`, `tgkill(2)`
- `getpid(2)`, `getppid(2)`, `ptrace(2)`, `getpgrp(2)`, `setpgrp(2)`

9 Submission Checklist

- Submission has a valid makefile with the rule for the target `animate.o` and compiles using *make animate.o*.
- Program is organised into multiple source and header files (for larger programs).
- Not include any object file, binary, or junk data in your git repo.
- If you have used AI, `references.zip` formatted according to EdStem slides submitted with source code.

Any memory leaks result in a 30% deduction after all other marks have been calculated, but will not decrease your mark below 40%.

A Version history

We aim to resolve all spec updates within the first 3-5 days.

B API documentation

The API documentation has been attached on the following pages for your convenience.

PointerPro Animate

Generated by Doxygen 1.9.8

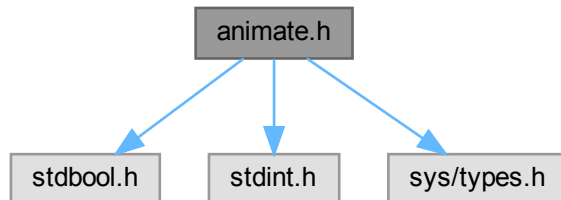
Chapter 1

File Documentation

1.1 animate.h File Reference

```
#include <stdbool.h>
#include <stdint.h>
#include <sys/types.h>
```

Include dependency graph for animate.h:



Typedefs

- typedef uint32_t [color_t](#)
Representation of a pixel.
- typedef void(* [animate_fn](#)) (void *priv, ssize_t *x, ssize_t *y, float t)
Custom animation function for optional extension.

Functions

- struct canvas * [animate_create_canvas](#) (size_t height, size_t width, [color_t](#) background_color)
Create the canvas on which our animation will be built.
- void [animate_destroy_canvas](#) (struct canvas *canvas)
Clean up a canvas.
- struct sprite * [animate_create_sprite](#) (const char *file)

- Create a sprite from the provided input bitmap file.*
 - struct sprite * [animate_create_rectangle](#) (size_t width, size_t height, color_t c, bool filled)
- Create a sprite that represents a rectangle.*
 - struct sprite * [animate_create_circle](#) (size_t radius, color_t c, bool filled)
- Create a sprite that represents a circle.*
 - bool [animate_destroy_sprite](#) (struct sprite *sprite)
- Frees all memory associated with a sprite.*
 - struct sprite_placement * [animate_place_sprite](#) (struct canvas *canvas, struct sprite *sprite, ssize_t x, ssize_t y)
- Places a sprite on the top layer of the canvas.*
 - void [animate_placement_up](#) (struct sprite_placement *sprite_placement)
- Raise a sprite one level towards the top layer.*
 - void [animate_placement_down](#) (struct sprite_placement *sprite_placement)
- Lower a sprite one level towards the bottom layer.*
 - void [animate_placement_top](#) (struct sprite_placement *sprite_placement)
- Raise the sprite to the top-most layer.*
 - void [animate_placement_bottom](#) (struct sprite_placement *sprite_placement)
- Lower the sprite to the bottom-most layer.*
 - void [animate_destroy_placement](#) (struct sprite_placement *sprite_placement)
- Removes the placement from its associated canvas and free resources.*
 - void [animate_set_animation_params](#) (struct sprite_placement *sprite_placement, ssize_t vx, ssize_t vy, ssize_t ax, ssize_t ay)
- Configure sprite animation.*
 - void [animate_set_animation_funtion](#) (struct sprite_placement *sprite_placement, [animate_fn](#), void *priv)
- size_t [animate_frame_size_bytes](#) (struct canvas *canvas)
- Return the size in bytes of an animation frame.*
 - void [animate_generate_frame](#) (struct canvas *canvas, size_t frame, size_t frame_rate, void *buf)
- Generate a frame of the animation.*

1.1.1 Typedef Documentation

1.1.1.1 animate_fn

```
typedef void(* animate_fn) (void *priv, ssize_t *x, ssize_t *y, float t)
```

Custom animation function for optional extension.

Parameters

<i>priv</i>	a handle to the private data provided when the function was registered
<i>x</i>	a reference to the starting position. On return, this should contain the target sprite position for the frame in question.
<i>y</i>	a reference to the starting position. On return, this should contain the position for the frame in question.
<i>t</i>	the current time since the start of the animation.

1.1.1.2 color_t

```
typedef uint32_t color_t
```

Representation of a pixel.

1.1.2 Function Documentation

1.1.2.1 animate_create_canvas()

```
struct canvas * animate_create_canvas (
    size_t height,
    size_t width,
    color_t background_color )
```

Create the canvas on which our animation will be built.

Parameters

<i>height</i>	height of the drawing area in pixels
<i>width</i>	width of the drawing area in pixels
<i>background_color</i>	the colour to use when no sprites overlap a pixel.

Returns

Abstract data type for the canvas. The memory associated this struct is allocated by this function and cleaned up by the corresponding destroy function

1.1.2.2 animate_create_circle()

```
struct sprite * animate_create_circle (
    size_t radius,
    color_t c,
    bool filled )
```

Create a sprite that represents a circle.

We currently only support filled circles. The circle pixels can be determined by the following formula: $x*x + y*y < r*r$ Where x is the x offset from the center of the circle y is the y offset from the center of the circle r is the radius of the circle

Parameters

<i>radius</i>	the radius of the circle
<i>c</i>	the colour of the circle
<i>filled</i>	reserved for future use

Returns

A handle to the created sprite

1.1.2.3 animate_create_rectangle()

```
struct sprite * animate_create_rectangle (
    size_t width,
```

```
size_t height,  
color_t c,  
bool filled )
```

Create a sprite that represents a rectangle.

Parameters

<i>width</i>	the width of the rectangle
<i>height</i>	the height of the rectangle
<i>c</i>	the colour of the rectangle
<i>filled</i>	true if the rectangle should be filled, otherwise false

Returns

A handle to the created sprite

1.1.2.4 `animate_create_sprite()`

```
struct sprite * animate_create_sprite (  
    const char * file )
```

Create a sprite from the provided input bitmap file.

The file format can be assumed to be ARGB32. The bitmap content may be loaded only on first access, however it must only be loaded once. NOTE: the order of image rows differs to what we expect. The first pixel in the bmp file format is on the bottom left while the last pixel is the top right. For simplicity, reverse the order of rows while loading the bitmap.

Parameters

<i>file</i>	the name of the bitmap file to be loaded
-------------	--

1.1.2.5 `animate_destroy_canvas()`

```
void animate_destroy_canvas (  
    struct canvas * canvas )
```

Clean up a canvas.

Frees all memory associated with the canvas, including placements, but not sprites

Parameters

<i>canvas</i>	the canvas to clean up
---------------	------------------------

1.1.2.6 animate_destroy_placement()

```
void animate_destroy_placement (
    struct sprite_placement * sprite_placement )
```

Removes the placement from its associated canvas and free resources.

Parameters

<i>sprite_placement</i>	a handle to the placement
-------------------------	---------------------------

1.1.2.7 animate_destroy_sprite()

```
bool animate_destroy_sprite (
    struct sprite * sprite )
```

Frees all memory associated with a sprite.

Parameters

<i>sprite</i>	the sprite to clean up
---------------	------------------------

Returns

0 on success, otherwise an error occurred. The only error possible is that the sprite is still in use

1.1.2.8 animate_frame_size_bytes()

```
size_t animate_frame_size_bytes (
    struct canvas * canvas )
```

Return the size in bytes of an animation frame.

This is useful for the caller to determine how much memory to allocate when combining sprites into a frame.

Parameters

<i>canvas</i>	the canvas in question
---------------	------------------------

Returns

the size, in bytes, of a frame for this canvas

1.1.2.9 animate_generate_frame()

```
void animate_generate_frame (
    struct canvas * canvas,
```

```

size_t frame,
size_t frame_rate,
void * buf )

```

Generate a frame of the animation.

Parameters

<i>canvas</i>	the canvas describing the animation
<i>frame</i>	the frame number (starting from 0)
<i>frame_rate</i>	the number of frames per second in the animation
<i>buf</i>	a buffer provided by the user that is large enough to hold a frame. This buffer will contain the frame data on return.

1.1.2.10 `animate_place_sprite()`

```

struct sprite_placement * animate_place_sprite (
    struct canvas * canvas,
    struct sprite * sprite,
    ssize_t x,
    ssize_t y )

```

Places a sprite on the top layer of the canvas.

Parameters

<i>canvas</i>	the canvas to place the sprite on
<i>sprite</i>	the sprite to place
<i>x</i>	the x coordinate of the canvas to which the top left pixel of the sprite should be placed
<i>y</i>	the y coordinate of the canvas to which the top left pixel of the sprite should be placed

Returns

A handle to a sprite placement.

1.1.2.11 `animate_placement_bottom()`

```

void animate_placement_bottom (
    struct sprite_placement * sprite_placement )

```

Lower the sprite to the bottom-most layer.

Parameters

<i>sprite_placement</i>	a handle to the placement
-------------------------	---------------------------

1.1.2.12 `animate_placement_down()`

```
void animate_placement_down (
    struct sprite_placement * sprite_placement )
```

Lower a sprite one level towards the bottom layer.

Parameters

<i>sprite_placement</i>	a handle to the placement
-------------------------	---------------------------

1.1.2.13 `animate_placement_top()`

```
void animate_placement_top (
    struct sprite_placement * sprite_placement )
```

Raise the sprite to the top-most layer.

Parameters

<i>sprite_placement</i>	a handle to the placement
-------------------------	---------------------------

1.1.2.14 `animate_placement_up()`

```
void animate_placement_up (
    struct sprite_placement * sprite_placement )
```

Raise a sprite one level towards the top layer.

Parameters

<i>sprite_placement</i>	a handle to the placement
-------------------------	---------------------------

1.1.2.15 `animate_set_animation_funtion()`

```
void animate_set_animation_funtion (
    struct sprite_placement * sprite_placement,
    animate_fn ,
    void * priv )
```

@breif [Optional extension] define animation using the provided function pointer

Parameters

<i>sprite_placement</i>	a handle to the sprite placement
<i>animate_fn</i>	a custom function to call when determining the position of the sprite in each frame.
<i>priv</i>	private data that is passed to the provided function on call.
<i>ax</i>	the acceleration in the x direction
<i>ay</i>	the acceleration in the y direction

1.1.2.16 animate_set_animation_params()

```
void animate_set_animation_params (
    struct sprite_placement * sprite_placement,
    ssize_t vx,
    ssize_t vy,
    ssize_t ax,
    ssize_t ay )
```

Configure sprite animation.

Currently, we support only physics simulation (velocity and acceleration)

Parameters

<i>sprite_placement</i>	a handle to the sprite placement
<i>vx</i>	the initial velocity in the x direction
<i>vy</i>	the initial velocity in the y direction
<i>ax</i>	the acceleration in the x direction
<i>ay</i>	the acceleration in the y direction

1.2 animate.h

[Go to the documentation of this file.](#)

```
00001 #ifndef ANIMATE_H
00002 #define ANIMATE_H
00003
00004 #include <stdbool.h> /* for bool */
00005 #include <stdint.h> /* for uint32_t */
00006 #include <sys/types.h> /* For size_t/ssize_t */
00007
00015 struct sprite;
00016 struct canvas;
00017 struct sprite_placement;
00018
00022 typedef uint32_t color_t;
00023
00035 typedef void (*animate_fn)(void* priv, ssize_t* x, ssize_t* y, float t);
00036
00037 // Inline functions are often preferred over macros as types will be checked
00041 static inline color_t animate_color_rgb(unsigned r, unsigned g, unsigned b) {
00042     return ((r & 0xff) << 16) | ((g & 0xff) << 8) | ((b & 0xff) << 0);
00043 }
00044
00048 static inline color_t animate_color_argb(unsigned a,
00049     unsigned r, unsigned g, unsigned b) {
00050     return animate_color_rgb(r, g, b) | ((a & 0xff) << 24);
00051 }
00052
00063 struct canvas* animate_create_canvas(size_t height, size_t width,
00064     color_t background_color);
00065
00073 void animate_destroy_canvas(struct canvas* canvas);
00074
00086 struct sprite* animate_create_sprite(const char* file);
00087
00098 struct sprite* animate_create_rectangle(size_t width, size_t height, color_t c,
00099     bool filled);
00100
00117 struct sprite* animate_create_circle(size_t radius, color_t c, bool filled);
00118
00126 bool animate_destroy_sprite(struct sprite* sprite);
00127
00139 struct sprite_placement* animate_place_sprite(struct canvas* canvas,
00140     struct sprite* sprite,
00141     ssize_t x, ssize_t y);
00142
```

```
00148 void animate_placement_up(struct sprite_placement* sprite_placement);
00149
00155 void animate_placement_down(struct sprite_placement* sprite_placement);
00156
00162 void animate_placement_top(struct sprite_placement* sprite_placement);
00163
00169 void animate_placement_bottom(struct sprite_placement* sprite_placement);
00170
00176 void animate_destroy_placement(struct sprite_placement* sprite_placement);
00177
00178
00189 void animate_set_animation_params(struct sprite_placement* sprite_placement,
00190                                   ssize_t vx, ssize_t vy,
00191                                   ssize_t ax, ssize_t ay);
00192
00204 void animate_set_animation_funtion(struct sprite_placement* sprite_placement,
00205                                    animate_fn, void* priv);
00206
00215 size_t animate_frame_size_bytes(struct canvas* canvas);
00216
00227 void animate_generate_frame(struct canvas* canvas,
00228                             size_t frame, size_t frame_rate,
00229                             void* buf);
00230
00231
00232 #endif /* ANIMATE_H */
```

